A Generative-AI Multi-Agent System for complex problem solving through Reasoning Chains and Formal Theorem Verification*

*Note: Sub-titles are not captured in Xplore and should not be used

1st Andres Felipe Mongui Torres
systems and computer engineering
Universidad Pedagogica Y Tecnologica de Colombia UPTC
Sogamoso- Boyaca
email address or ORCID

2nd Nicolas Samuel Tinjaca Topia
systems and computer engineering
Universidad Pedagogica Y Tecnologica de Colombia UPTC)
Sogamoso-Boyaca
email address or ORCID

Abstract

Index Terms

I. Introduction

Currently, the general-purpose large language models (LLMs) solve a large fraction of mathematics problems, and in 2025 some systems reached gold-medal level at the international Mathematical Olympiad [11]. However, this kind of reasoning offer no guarantee of logical validity, this is because a chain of thought [2] can be fluent yet still incorrect. there are also proof assistants such as Lean 4 [14] have the opposite side of the problem, they checks every step mechanically, but the process of writing formal proofs by hand is slow and demands expertise, finally there are also systems like LeanDojo [8], AlphaProof [10] and DeepSeek-prover-V2 [9] have combined those both regimes, but using computational budgets far beyond those of a academic course. Due to this, exist a need of for an auditable and resource-bounded pipeline in which every accepted answer is backed by a kernel verdict.

This project want to ask to what extent a multi-agent architecture that has separated the generation of proof hypotheses, and supported by embedding-based retrieval improves the success rate, efficiency and reliability with respect to a single LLM reasoning in natural language and a single orchestrated as a state graph specialized agents. To make this possible, this project aims to specify the architecture with typed interface contracts and implement a retrieval module over the Mathlib library.

The scope of the project covers algebra, number theory and elementary combinatorics problems already formalized in miniF2F [12] and PutnamBench [13], by using LLMs accesed through an API, finally this project has some limitations, for example the kernel certifies that a proof matches its Lean statement, but doesn't verify that the statement faithfully captures the original problem; also an evaluation ser of nearly 30 or 60 problems only allows large difference between configurations to reach statistical significance

Identify applicable funding agency here. If none, delete this.

II. Background

Writing mathematical language in a formal, correct, and efficient manner is a costly process that has traditionally required highly trained experts skilled enough to translate mathematical statements into a rigorously formal syntax. Efforts are underway to reduce this cost and difficulty through the use of artificial intelligence models capable not only of solving this problem based on expert instructions but also of learning to perform the translation autonomously, leveraging datasets and constant feedback.

Over the last decade, the development, research, and study of artificial intelligence applied to solving mathematical problems have advanced exponentially, reaching the point where generative language models are now being integrated with formal verification systems.

The Lean 4 system has emerged as one of the most effective environments for the formal verification of mathematical problems. Its efficiency stems from a mechanism based on representing statements and proofs in a formally verifiable manner, guaranteeing the logical correctness of results when required. Lu et al. [26] introduce the concept of "auto-formalization" to describe the ability of language models to automatically transform mathematical expressions written in natural language into formal code using Lean 4.

Theorem verification in Lean 4 relies on the models' ability to generate hypotheses and accurately identify lemmas and premises relevant and useful for the analysis. Premises are understood as pre-existing, verified mathematical results that the Mathlib library uses as a foundation for constructing proofs. Gao et al. [27] introduce LeanSearch v2, a premise retrieval system that combines embedding techniques—representing complex information as vectors so that semantic relationships correspond to vector proximity—with iterative cycle techniques to enhance performance and increase the success rate of verified proofs. Model efficiency during analysis is also influenced by the benchmarking approach; Liu et al. [28] present benchmarks—standardized datasets used to measure and compare AI model performance—featuring two testing modes. The "Easy Mode" embeds the final answer within the formal statement to be constructed, simplifying the task for the model but failing to provide a realistic assessment of system capabilities. Conversely, "Hard Mode" requires the model to first construct the solution before building the formal proof; this creates a more rigorous, realistic environment that yields accurate measures of model capability. This is confirmed by their results: the proposed framework formally proves 36 PutnamBench theorems in Hard Mode, and LLMs exceed 80 percent answer accuracy on problems where formal provers achieve under 10 percent.

Although the tests conducted are effective and many LLMs have been developed to translate mathematical proofs from natural to formal language, most testing is performed on filtered, raw, or curated datasets; this raises questions about the models' efficiency in more realistic environments. Gui et al. [29] present a study on how these models perform when faced with perturbed data—specifically, scenarios requiring the system to maintain data coherence and correctness regardless of phrasing, symbols, or value modifications, while automatically correcting any errors. The analysis utilized MiniF2F and MATH-500, measuring the stability of the applied corrections and their fidelity to the original data.

- mini F2F: A dataset of Olympiad-level math problems translated into formal format using Lean 4. It serves as a benchmark for measuring whether an AI can generate verifiable mathematical code (formal proofs) rather than just a textual analysis.
- MATH-500: 500 highly difficult problems—covering algebra, calculus, geometry, etc.—used to determine whether AI can arrive at the correct numerical answer and to verify reasoning.

The results indicate that the models struggle significantly with highly variable data and fail to handle it effectively, demonstrating that current models are inadequate for scenarios involving substantial natural language variation; Consequently, the research proposes not only correcting the datasets but also enhancing the robustness of the models' application to them.

Several models currently exist that can decompose complex problems into independently verifiable objectives. Ren et al. [9] demonstrate the use of reinforcement learning to generate and verify these sub-objectives within Lean 4, showing how this method significantly improves process efficiency and yields higher success rates compared to standard approaches. Results include an 88 percent success rate on the miniF2F benchmark, validating the efficiency of recursive decomposition. The study confirms that analyzing the problem structure, planning and creating sub-objectives, and focusing on them constitutes a more efficient formalization strategy. However, applying this technique introduces risks during sub-objective formulation; proofs may be technically correct yet fail to align with the original problem statement, necessitating feedback mechanisms to ensure the fidelity of the formalizations [9].

III. State of the art

In the field of AI-assisted mathematical analysis and reasoning, benchmarks are fundamental to measuring the progress of theorem-proving systems. A major limitation is that some datasets focus exclusively on basic-level algebra and calculus [29]. It has also been noted that current LLM-based systems are far from achieving 100 percent effectiveness in problem-solving within realistic scenarios, given that success rates have been found to be below 20 percent.

Significant progress has been made in the field of auto-formalization applied to formal mathematics. While LLMs such as GPT or Claude can translate mathematical statements into formal languages, they exhibit significant error rates; conversely, specialized models like DeepSeek and Qwen-Math achieve higher accuracy but rely heavily on well-structured, curated datasets [31]. An analysis of MiniF2F, ProofNet, FormalMATH, and PutnamBench reveals that these datasets vary in both difficulty and thematic coverage. The shift toward more realistic scenarios highlights the need to enhance model robustness in real-world contexts and ensure high scalability for increasingly complex problems [31].

The analysis, which was optimally applied, has revealed the need to integrate informal reasoning with formal verification; improvements in performance and efficiency are observed when LLMs generate hypotheses by leveraging context retrieval via embeddings and RAG. Greater effectiveness is evident in multi-agent systems that combine premise retrieval with automated verification, with each agent fulfilling a specific role [32].

In the realm of automated theorem proving, the interaction between models and formal systems presents a challenge that Zhou et al. [32] address by providing a framework allowing LLMs to interact directly with Lean 4 without requiring training. This approach yields improved efficiency through reflective decomposition and interactive proof processes, leveraging the flexibility of LLMs alongside Lean 4's structure to generate verifiable proofs. Results demonstrate a 95.9 percent success rate on miniF2F-test when combining informal reasoning with formal verification. Tang [33] offers another example of this integration, proving effective even in highly specialized fields—such as geometry problems at the International Olympiad level—by enabling the agent to formulate hypotheses in natural language and convert them into verifiable formal proofs.

While informal reasoning verified using Lean 4 is efficient, it presents challenges—such as identifying key concepts, analyzing the relationships between them, and formalizing the results. Li et al. [35] introduce KG-Prover as a solution that employs knowledge graphs to significantly expand an LLM's context during proof generation, leveraging the model's natural language reasoning capabilities. This approach represents the graph using definitions, axioms, theorems, and their respective relationships, dividing the process into four stages: analyzing the graph, generating informal proofs, self-formalizing in Lean 4, and iteratively verifying the results—thereby scaling the model's mathematical reasoning capacity. Yin and Gao [36] present LeanNavigator, a graph exploration system that generates proofs, demonstrating high performance and increased result accuracy. Also introduced is Theorem Graph—a graph of informal statements designed to facilitate structured study and mathematical reasoning; it uses LLMs to validate data coverage and accuracy, representing data via embeddings and comparing it with previous systems. This bridges the gap between informal and formal approaches, though information loss may occur during analysis, or not all necessary nodes may be covered. Beyond the aforementioned methods, Kripner et al. [37] present the "white-box" approach, in which the system incrementally processes the Lean identifier, analyzes each intermediate state, and splits proofs into independent branches. This creates subsets that allow for the partial or total reuse of results and enables parallel searches, thereby improving process efficiency.

IV. Materials and Methods

The list of the resources used in this project is in Table 1 with his category and the role within the project's development.

For this project we followed an iterative Design Science Research process [22], in witch the multi agent system is designed to resolve the needs described in the section 1. Each component is validated individually before the integration into the complete pipeline, thanks to this, the theorical foundations of reasoning chains, retrieval and formal verification are connected to a measurable implementation, the phases of this process follows the next steps:

a) Foundation and requirements specification.: As a first phase, we identify the system's functional and non-functional requirements, defining what the pipeline must do and the quality it must possess. such as reliability, reproducibility and a bounded number of LLM calls per problem

b) Architecture design.: One we defined all the requirements, we made the design of the state graph of the orchestrator, the interface between agents and the mathematical model

TABLE I: Materials: data, models and software.

Category	Resource	Role in the project
Benchmarks	miniF2F (Lean 4 port) [12]; PutnamBench [13]	Source of formally stated problems; primary and secondary evaluation pools.
Formal system	Lean 4 [14] + Mathlib [15] (Apache 2.0)	Trusted kernel (verifier) and lemma corpus for retrieval.
Lean interface	Lean REPL / `lean -server`; Pantograph [16]	Programmatic compilation, goal-state inspection and error traces.
LLMs (reasoning)	Frontier model via API (GPT / Claude / Gemini family, version pinned)	Planner, Generator and Critic feedback.
LLMs (formal)	DeepSeek-Prover-V2-7B [9]; Qwen2.5-Math	Autoformalizer (sketch $\rightarrow$ Lean tactics); local fallback.
Embeddings	BGE / E5 family [18] or `text-embedding-3`	Dense vector representation of subgoals and Mathlib declarations.
Vector index	FAISS [19] (exact inner-product index)	k-nearest-neighbour search over the declaration corpus.
Orchestration	LangGraph (typed state graph with checkpoints)	Implements the state machine of the orchestrator.
Validation	Python `jsonschema`, `pytest`	Contract tests and regression suite.
Publication	Hugging Face Hub (dataset, Space with Gradio)	Reproducible release and demo.

c) Incremental implementation.: The system is built starting from the single agent baselines incorporating the planner and Generator, the autoformalizer and verifier connected to Lean 4 and Mathlib, and finally the embedding based module, after each incorporation all the components are correctly checked using unit test, and contract test againts their JSON schemas

d) Evaluation set curation.: In the next step several problem sets are selected from miniF2F, and organized by difficulty and mathematical area, and finally split into disjoint development to avoid overfitting the prompts to the test problems

e) Experimentation.: Once we have the complete pipeline and the problem sets a set of 30 to 60 problems—already formalized in Lean 4—is selected from the public test sets miniF2F and PutnamBench, we run five configurations of the system on the test problems, we save everything: the solution, the number of calls, and the time taken

f) Analysis of results.: Once we finish the experimentation phase, all the configurations are compared using metrics, and failed attempts are classified according the error taxonomy

g) Packaging and documentation.: Finally, the general system and the experimental result are documented and published in hugging face with all the tecnical documentation

A. Complete model: mathematical formalization

This section formalises every stage of the pipeline.

TABLE II: Main notation.

Symbol	Meaning
x, φ	Natural-language problem and its Lean 4 statement (a proposition)
$\mathscr{L}$	Set of Lean 4 source strings (candidate proofs)
$\mathscr{V}$	Lean kernel verification function
σ	Proof skeleton produced by the Planner
$g_i = (\Gamma_i \vdash \psi_i)$	i-th subgoal: local context Γ_i and goal proposition ψ_i
$\mathscr{M} = \{m_j\}_{j=1}^{\lvert\mathscr{M}\rvert}$	Corpus of Mathlib declarations
$E : \Sigma^* \rightarrow \mathbb{R}^d$	Embedding encoder (Transformer)
$R_k(g)$	Top-k retrieved lemmas for subgoal g
s, c	Natural-language proof sketch and Lean candidate code
τ	Error trace returned by the verifier
$k_{\max}$, $R_{\max}$, N	Retries per subgoal, maximum replans, self-consistency samples
$B_{\max}$	Hard cap on LLM calls per problem

1) Problem and verification oracle: Let φ be a Lean 4 proposition with hypotheses and $\mathscr{L}$ the set of Lean source strings. The Lean kernel is modelled as a total, deterministic function

$$\mathscr{V} : \Phi \times \mathscr{L} \longrightarrow \{\top, \bot\} \times \mathscr{T}, \qquad \mathscr{V}(\varphi, c) = (v, \tau), \tag{1}$$

where $v = \top$ if and only if c elaborates to a term t with $\vdash t : \varphi$, and $\tau \in \mathscr{T}$ is the (possibly empty) list of diagnostic messages. Because a proof may compile while still containing placeholders or non-standard axioms, we define the *acceptance predicate*

$$\mathrm{Acc}(\varphi, c) \iff \mathscr{V}(\varphi, c)_1 = \top \wedge \texttt{sorry} \notin c \wedge \mathrm{Ax}(c) \subseteq \mathscr{A}_0, \tag{2}$$

where $\mathrm{Ax}(c)$ is the set of axioms reported by `#print axioms` and $\mathscr{A}_0 = \{$`propext`, `Classical.choice`, `Quot.sound`$\}$ is the standard axiom base of Mathlib. A problem is reported as *solved* only if $\mathrm{Acc}(\varphi, c^\star)$ holds for the final assembled proof $c^\star$ (RNF-1).

2) Subgoal decomposition (Planner): The Planner maps (x, φ) to a *skeleton* σ: a Lean proof of φ in which n intermediate facts are introduced as `have` $h_i : \psi_i$ `:= sorry`. Each hole defines a subgoal

$$g_i = (\Gamma_i \vdash \psi_i), \qquad \Gamma_i = \Gamma_\varphi \cup \{h_1 : \psi_1, \ldots, h_{i-1} : \psi_{i-1}\}, \qquad i = 1, \ldots, n \le n_{\max}, \tag{3}$$

where Γ_φ are the hypotheses of φ. The skeleton is itself checked by the kernel in *sorry-tolerant* mode: it is *valid* if $\mathscr{V}(\varphi, \sigma)$ reports no error other than the n `sorry` warnings. This verifies that the decomposition is logically sufficient before any subgoal is attempted.

Lemma 1 (Compositionality)**.** *Let σ be a valid skeleton for φ with subgoals $g_1, \ldots, g_n$, and let π_i be a proof with $\mathrm{Acc}(g_i, \pi_i)$ for every i. Then $c^\star = \sigma[\pi_1/\texttt{sorry}_1, \ldots, \pi_n/\texttt{sorry}_n]$ satisfies $\mathrm{Acc}(\varphi, c^\star)$, provided no elaboration side-effects (e.g., timeouts) occur.*

Sketch. Each π_i is typed in context Γ_i, which is exactly the context in which the i-th hole of σ is elaborated. Substituting a well-typed term for a hole of the same type preserves well-typedness (substitution lemma of dependent type theory), and no `sorry` or new axiom is introduced. The side condition is checked empirically by re-verifying $c^\star$ in state ASSEMBLE. □

Remark 1. The final re-verification makes the system sound even if the lemma's side condition fails: a bug in any agent can cause a *failure*, never a false *acceptance*. The Planner's subgoals may be mis-formalised (e.g., too weak or false), but this only reduces the success rate.

The Planner uses chain-of-thought [2] and, optionally, a tree-of-thought [4] breadth b: it proposes b candidate strategies, scores each one with the LLM, and keeps the highest-scoring skeleton that is valid.

3) Embeddings and retrieval (RAG): Each Mathlib declaration m_j is rendered as a text $t(m_j)$ = *fully qualified name* $\|$ *type signature* $\|$ *docstring*. Following retrieval-augmented generation [17], a Transformer encoder [1] E maps texts to $\mathbb{R}^d$ and vectors are ℓ_2-normalised:

$$\hat{e}(u) = \frac{E(u)}{\|E(u)\|_2}, \qquad \mathrm{sim}(q, m_j) = \cos\big(E(q), E(t(m_j))\big) = \hat{e}(q)^\top \hat{e}\big(t(m_j)\big) \in [-1, 1]. \tag{4}$$

For a subgoal g, the query $q(g)$ is the pretty-printed goal $\Gamma \vdash \psi$ plus, on a retry, the missing identifier reported in τ. The retrieved context is

$$R_k(g) = \operatorname*{arg\,top\text{-}k}_{m_j \in \mathscr{M}} \mathrm{sim}\big(q(g), m_j\big). \tag{5}$$

The corpus embeddings form a matrix $\mathbf{M} \in \mathbb{R}^{|\mathscr{M}| \times d}$ computed once; the query cost is one encoder call plus an exact inner-product search $\mathbf{M}\hat{e}(q)$ in FAISS, i.e., $\mathscr{O}(|\mathscr{M}|\, d)$. Optionally, to reduce redundancy among near-duplicate lemmas, the list is re-ranked with Maximal Marginal Relevance [20]:

$$m^\star = \operatorname*{arg\,max}_{m \in C \setminus S} \Big[\lambda\, \mathrm{sim}(q, m) - (1 - \lambda) \max_{m' \in S} \mathrm{sim}(m, m')\Big], \qquad \lambda \in [0, 1]. \tag{6}$$

a) Retrieval validation.: Before integration, retrieval is evaluated on a reference set Q of (goal, gold-lemma set G_q) pairs extracted from existing Mathlib proofs:

$$\text{Recall@}k = \frac{1}{|Q|}\sum_{q\in Q} \mathbb{1}\big[R_k(q)\cap G_q \neq \emptyset\big], \qquad \text{MRR} = \frac{1}{|Q|}\sum_{q\in Q}\frac{1}{\text{rank}_q}, \tag{7}$$

where rank_q is the position of the first gold lemma. The value of k is chosen on the development split by maximising Recall@k subject to the context-length budget of the Generator.

4) Generation with self-consistency: The Generator is a conditional distribution over natural-language sketches,

$$s \sim p_\theta\big(\cdot \mid g, R_k(g), f\big), \tag{8}$$

where f is the Critic's feedback (empty on the first attempt). Following self-consistency [6], N sketches $s^{(1)},\dots,s^{(N)}$ are sampled at temperature $T>0$. Each sketch carries a *strategy tag* $\ell(s)\in\mathscr{S}$ (e.g., induction, case analysis, modular arithmetic, AM–GM). The selected sketch belongs to the modal strategy:

$$\ell^\star = \arg\max_{\ell\in\mathscr{S}} \sum_{r=1}^{N} \mathbb{1}\big[\ell(s^{(r)})=\ell\big], \qquad \hat{s} = \arg\max_{s^{(r)}:\,\ell(s^{(r)})=\ell^\star} \log p_\theta\big(s^{(r)}\mid\cdot\big), \tag{9}$$

with ties broken by the sequence log-probability (or by an LLM self-score when log-probabilities are not exposed by the API). The agreement ratio $\alpha = \max_\ell \frac{1}{N}\sum_r \mathbb{1}[\ell(s^{(r)})=\ell]$ is logged as a confidence signal but *never* used for acceptance.

5) Autoformalisation: The Autoformalizer maps the sketch to Lean code conditioned on the subgoal, the retrieved lemmas and the last error:

$$c \sim p_\psi\big(\cdot \mid g, \hat{s}, R_k(g), \tau_{\text{prev}}\big). \tag{10}$$

It is decoded with low temperature and a syntactic post-check (balanced blocks, only identifiers from $R_k(g)$, the local context or core Lean). If the post-check fails after one internal repair, it reports the status *formalization failed* (RF-4).

6) Critique and decision policy: When $\mathscr{V}(g_i,c)_1 = \bot$, the Critic first classifies the trace with a deterministic rule-based classifier $\kappa:\mathscr{T}\to\mathscr{E}$, where

$$\mathscr{E} = \{\textsf{syntax},\ \textsf{missing_lemma},\ \textsf{tactic_failure},\ \textsf{misformalization},\ \textsf{timeout},\ \textsf{planning}\}. \tag{11}$$

For example, *unknown identifier* $\mapsto$ missing_lemma; *unexpected token* $\mapsto$ syntax; *deterministic timeout / maximum heartbeats* $\mapsto$ timeout; *unsolved goals / type mismatch* $\mapsto$ tactic_failure; a subgoal refuted or failed with $k_{\max}$ distinct strategies $\mapsto$ planning. Let a_i be the attempts spent on subgoal i, r the replans performed and b the LLM calls consumed. The control action is a deterministic function

$$\pi(e,a_i,r,b) = \begin{cases} \textsf{abandon} & \text{if } b\ge B_{\max} \ \text{ or } \ \big(a_i\ge k_{\max}\wedge r\ge R_{\max}\big),\\ \textsf{replan} & \text{if } \big(a_i\ge k_{\max}\vee e=\textsf{planning}\big)\wedge r<R_{\max},\\ \textsf{retry}(e) & \text{otherwise,}\end{cases} \tag{12}$$

where $\textsf{retry}(e)$ routes to FORMALIZE if $e\in\{\textsf{syntax}\}$ (the sketch is kept), to RETRIEVE if $e=\textsf{missing_lemma}$ (query expanded and k increased), and to GENERATE otherwise. Only the natural-language feedback f is produced by an LLM, following Self-Refine [5] and ReAct [3]; control flow is deterministic, which makes termination provable.

Proposition 1 (Termination and cost bound)**.** *Every execution reaches* DONE *or* FAIL*, and the number of LLM calls per problem satisfies*

$$C \ \le\ \min\Big\{B_{\max},\ (R_{\max}+1)\big(b_P + n_{\max}k_{\max}(N+2)\big)\Big\}, \tag{13}$$

where b_P is the number of LLM calls of one planning step ($b_P=1$ for chain-of-thought, $b_P=b+1$ for tree-of-thought breadth b).

Sketch. Each attempt on a subgoal costs at most N Generator calls, one Autoformalizer call and one Critic call. Attempts per subgoal are bounded by $k_{\max}$ through Equation (12); a replan resets attempts but increments r, which

is bounded by $R_{\max}$; each plan has at most $n_{\max}$ subgoals. The Retriever and Verifier make no LLM calls. Hence the number of transitions is finite, and the explicit budget check $b \geq B_{\max}$ adds a hard cap. □

With the default configuration ($n_{\max} = 3$, $k_{\max} = 3$, $N = 3$, $R_{\max} = 1$, $b_P = 1$) the structural bound is $2(1+45) = 92$ calls; we set $B_{\max} = 40$ as a hard cap per problem and target the efficiency requirement RNF-3 of *at most 20 LLM calls on average per solved problem*, defined as $\bar{C}_{\text{solved}} = \sum_{x \in \mathscr{D}} C(x) / |\{x : \text{solved}\}|$ (failed problems' cost is included in the numerator).

7) Success model for bounded retries: To interpret the effect of retries, consider a reference model in which attempts on subgoal g_i are independent Bernoulli trials with success probability p_i. Then

$$P_i(k) = 1 - (1-p_i)^k, \qquad P_{\text{solve}}(k) = \prod_{i=1}^{n} \left[1 - (1-p_i)^k\right], \qquad \Delta_i(k) = P_i(k) - P_i(k-1) = p_i(1-p_i)^{k-1}. \tag{14}$$

Under independence the marginal gain of each additional retry decays geometrically. An observed gain *above* this curve is evidence that critic feedback makes attempts informative (positively dependent on the previous trace), which is exactly what the refinement-robustness metric (Section IV-A9) measures. The product form also shows why decomposition helps only if subgoals are easier than the whole: it requires $\prod_i P_i(k) > P_\varphi(k)$ at comparable cost.

8) Evaluation set and ablation configurations: The evaluation set $\mathscr{D}$ contains 45–54 problems (two-member team) stratified on a 3×3 grid of difficulty (easy, medium, hard) $\times$ area (algebra, number theory, combinatorics), with 5–6 problems per cell when available and proportional reallocation otherwise. $\mathscr{D}$ is split into disjoint development ($\approx 1/3$, prompt tuning and hyper-parameter selection) and test ($\approx 2/3$, touched only for final runs) partitions. Five configurations are compared on the test split:

C1. Baseline A: single LLM, direct prompting for a Lean 4 proof, no agents.

C2. Baseline B: single LLM with natural-language chain-of-thought, no formal verification.

C3. No-RAG: full pipeline with $R_k(g) = \emptyset$.

C4. No-Critic: full pipeline with $k_{\max} = 1$ and $R_{\max} = 0$.

C5. Full system: all agents active, $k_{\max} \in \{3, \ldots, 5\}$.

Every configuration is a flag set over the same LangGraph implementation, so they differ only in the ablated component.

9) Metrics:

a) Verified success rate and pass@k.: For configuration m, $\text{SR}_m = \frac{1}{|\mathscr{D}|} \sum_{x \in \mathscr{D}} \mathbb{1}[\text{Acc}(\varphi_x, c_x^\star)]$. With n independent runs per problem of which c_x succeed, pass@k is estimated without bias as [21]

$$\widehat{\text{pass@}k} = \frac{1}{|\mathscr{D}|} \sum_{x \in \mathscr{D}} \left[1 - \binom{n - c_x}{k} \Big/ \binom{n}{k}\right]. \tag{15}$$

b) Avoided false positives.: Let $\mathscr{B}^+$ be the problems that Baseline B self-assesses as correct. Each such natural-language proof is passed to the Autoformalizer–Verifier loop with the same budget, and

$$\text{FPR}_B = \frac{\left|\{x \in \mathscr{B}^+ : \neg\text{Acc}(\varphi_x, c_x)\}\right|}{|\mathscr{B}^+|}. \tag{16}$$

This is an upper bound on B's true false-positive rate, since a correct informal proof may still fail autoformalisation; a manual review of a sample separates both cases.

c) Efficiency.: LLM calls, retrieval calls, total tokens and wall-clock time per solved problem ($\bar{C}_{\text{solved}}$ as above).

d) Refinement robustness.: Empirical marginal gain $\hat{\Delta}(k) = \widehat{\text{SR}}(k) - \widehat{\text{SR}}(k-1)$ for $k = 1, \ldots, k_{\max}$, compared against the independence curve of Equation (14).

e) RAG contribution.: $\Delta_{\text{RAG}} = \text{SR}_{\text{C5}} - \text{SR}_{\text{C3}}$, tested with McNemar's test.

f) Error taxonomy.: Relative frequency of each class in $\mathscr{E}$ over all failed attempts, per configuration and per area.

B. System Design

All the system was organized as a state graph, in which each node corresponds to a specialized agent, each box is a state, and each state is handled by one agent, the process stars in PLAN, where the general problem is divided in smaller parts, then for each subgoal the system follows the next 4 steps: RETRIEVE search for useful lemmas in MathLib, GENERATE write the general idea of the proof, FORMALIZE turns that idea into Lean Code, VERIFY checks the code with Lean, in this node there are two options, if the code is correct the system moves to the next subgoal or ends in Done if all the subgoals were solved, but if the code ends with an error, the system goes to CRITIQUE each can try again from GENERATE, or stop in FAIL when there are no mare attempts left, all this process is describe in Figure 1.

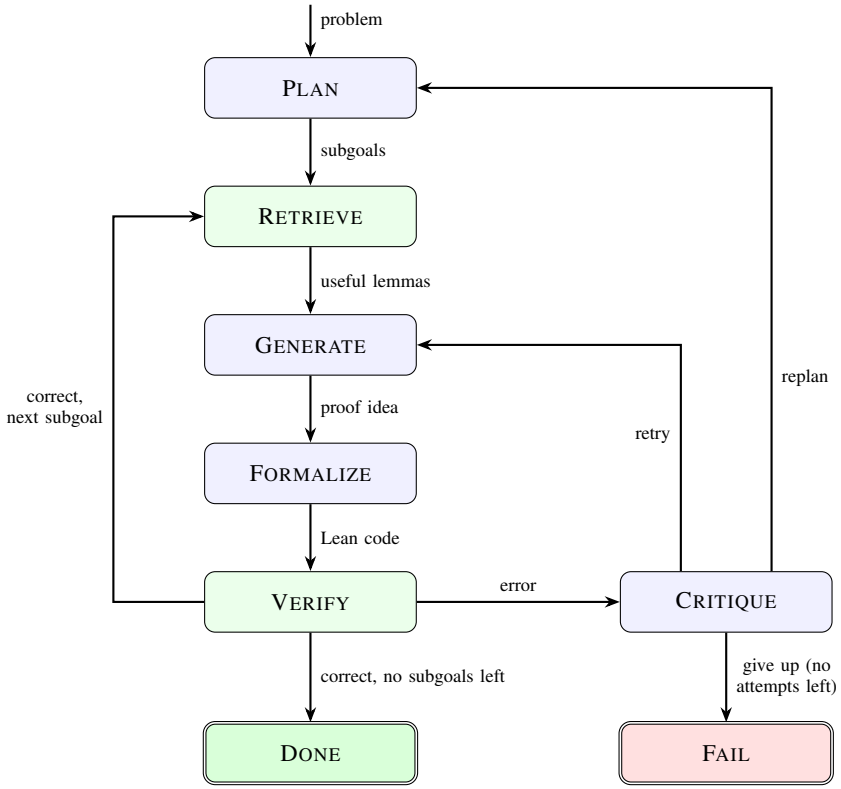


Fig. 1: State graph of the orchestrator. Blue states use an LLM, green states are deterministic and double borders mark the final states.

1) Multi-agent structure: The system is a multi-agent architecture made of six specialized agents coordinated by an orchestrator implemented in LangGraph. Each agent has a single role and only communicates with the others through the orchestrator, which receives the output of one agent, decides which state comes next and passes the necessary information to the next agent. Four agents use a language model (Planner, Generator, Autoformalizer and Critic), while the Retriever and the Verifier are deterministic, which guarantees that the final decision of whether a proof is correct never depends on an LLM. Table III shows the role of each agent, the state of the graph it is responsible for, what it receives and what it returns; this table also defines the interface contract between agents.

V. Development

VI. Conclusions

VII. Future Work

References

[1] A. Vaswani et al. Attention is all you need. In *NeurIPS*, 2017.

[2] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*, 2022.

[3] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023. arXiv:2210.03629.

TABLE III: Agents of the system and their interface contracts.

Agent	State	Role	Receives	Returns
Planner	PLAN	Divides the problem into subgoals (chain-of-thought).	The problem statement.	An ordered list of subgoals.
Retriever	RETRIEVE	Finds useful lemmas using embeddings.	A subgoal.	The most similar Mathlib lemmas.
Generator	GENERATE	Writes a proof idea (self-consistency).	The subgoal and the lemmas.	A proof idea in natural language.
Autoformalizer	FORMALIZE	Translates the idea into Lean.	The proof idea and the lemmas.	Candidate Lean 4 code.
Verifier	VERIFY	Checks the code with the Lean kernel.	The Lean code.	Accepted or rejected, plus the error message.
Critic	CRITIQUE	Analyses errors and decides what to do (Self-Refine).	The error and the previous attempts.	Retry, replan or give up.

[4] S. Yao et al. Tree of thoughts: Deliberate problem solving with large language models. In *NeurIPS*, 2023.
[5] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.
[6] X. Wang et al. Self-consistency improves chain of thought reasoning in language models. In *ICLR*, 2023.
[7] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. arXiv:2009.03393, 2020.
[8] K. Yang et al. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS Datasets and Benchmarks*, 2023.
[9] Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, et al. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. arXiv:2504.21801, 2025.
[10] Google DeepMind. AI achieves silver-medal standard solving International Mathematical Olympiad problems. Technical blog, 2024.
[11] Google DeepMind. Gemini Deep Think achieves gold-medal standard at the International Mathematical Olympiad. Technical blog, 2025.
[12] K. Zheng, J. M. Han, and S. Polu. MiniF2F: A cross-system benchmark for formal olympiad-level mathematics. In *ICLR*, 2022. arXiv:2109.00110.
[13] G. Tsoukalas et al. PutnamBench: Evaluating neural theorem-provers on the Putnam mathematical competition. In *NeurIPS Datasets and Benchmarks*, 2024.
[14] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE-28*, 2021.
[15] The mathlib Community. The Lean mathematical library. In *CPP*, 2020.
[16] L. Aniva et al. Pantograph: A machine-to-machine interaction interface for advanced theorem proving, high level reasoning, and data extraction in Lean 4. In *TACAS*, 2025.
[17] P. Lewis et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
[18] L. Wang et al. Text embeddings by weakly-supervised contrastive pre-training. arXiv:2212.03533, 2022.
[19] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.
[20] J. Carbonell and J. Goldstein. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *SIGIR*, 1998.
[21] M. Chen et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.
[22] A. R. Hevner, S. T. March, J. Park, and S. Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.
[23] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *JASA*, 22(158):209–212, 1927.
[24] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
[25] B. Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
[26] J. Lu, Y. Wan, Z. Liu, Y. Huang, J. Xiong, C. Liu, J. Shen, H. Jin, J. Zhang, H. Wang, Z. Yang, J. Tang, and Z. Guo. Process-driven autoformalization in Lean 4. arXiv:2406.01940, 2024.
[27] G. Gao, Z. Sun, J. Jiang, Y. Wang, J. Xu, P. Wu, B. Dai, and B. Dong. LeanSearch v2: Global premise retrieval for Lean 4 theorem proving. arXiv:2605.13137, 2026.

[28] C. Liu, Y. Yin, Y. Yuan, J. Xie, B. Li, S. Li, J. Shen, Y. Xu, L. Shang, and M. Zhang. Discover and Prove: An open-source agentic framework for Hard Mode automated theorem proving in Lean 4. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 117–133, 2026.
[29] Z. Gui, S. Yang, and Z. Shi. Evaluating the robustness of proof autoformalization in Lean 4. arXiv:2606.14867, 2026.
[30] Z. Shen, N. Huang, F. Yang, Y. Wang, G. Gao, T. Xu, J. Jiang, W. He, P. Yang, M. Sun, H. Ju, P. Wu, B. Dai, and B. Dong. REAL-Prover: Retrieval Augmented Lean Prover for Mathematical Reasoning. arXiv:2505.20613v3, 2025.
[31] K. Weng, L. Du, S. Li, W. Lu, H. Sun, H. Liu, and T. Zhang. Autoformalization in the Era of Large Language Models: A Survey. arXiv:2505.23486v2, 2025.
[32] Y. Zhou, J. Zhao, Y. Zhang, B. Wang, S. Wang, L. Chen, J. Wang, H. Chen, A. Jie, X. Zhang, H. Wang, L. Trung, R. Ye, P. N. Hoang, H. Zhang, P. Sun, and H. Li. Solving Formal Math Problems by Decomposition and Iterative Reflection. arXiv:2507.15225v1, 2025.
[33] X. Tang. Mathematical Formalized Problem Solving and Theorem Proving in Different Fields in Lean 4. Preprint, 2024. Available: https://github.com/txc701/open-source-code-for-AI4M
[34] H. Ju and B. Dong. AI for Mathematics: Progress, Challenges, and Prospects. arXiv:2601.13209v5, 2026.
[35] V. Li, Y. Fu, T. Knappe, K. Han, and K. Zhu. Scaling Natural-Language Graph-Based Test Time Compute for Automated Theorem Proving. arXiv:2503.11657, 2025.
[36] D. Yin and J. Gao. Generating Millions of Lean Theorems With Proofs By Exploring State Transition Graphs. arXiv:2503.04772, 2025.
[37] M. Kripner, M. Šustr, and M. Straka. LeanTree: Accelerating White-Box Proof Search with Factorized States in Lean 4. arXiv:2507.14722, 2025.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.